EEET2601 Engineering Computing 1

File IO

Quang Tran

RMIT UNIVERSITY

What is a file?

- A file is just a sequence of bytes stored on secondary storage devices such as disk
- Text file: contains the ASCII codes of characters (digits, letters, and other symbols), can be read with a basic text editor, e.g. C files, CSV files, etc.
- Binary file: non-text file that can't be read with a basic text editor, i.e. a.out, jpeg files, Word files, etc. Often contain metadata in addition to the content.
- We focus on working with text files in this course

↓
rendered
the content

File IO functions in <stdio.h>

Function	Description
FILE *fopen (const char *filename, const char *mode)	Open a file with a given mode. Return a FILE pointer on success and NULL on failure.
int fclose(FILE *fp)	Close a file. Return zero on success and EOF (usually -1) on failure.
int fgetc(FILE *fp)	Read and return a character from a file, or EOF on end of file or error.
int fputc(int c, FILE *fp)	Write a character to a file. Return the same character, or EOF if an error occurs.
int feof(FILE *fp)	Return a non-zero integer if the end of a file is reached, or zero otherwise.
rewind(FILE *fp)	Set the file position to the beginning of the file.

end of file

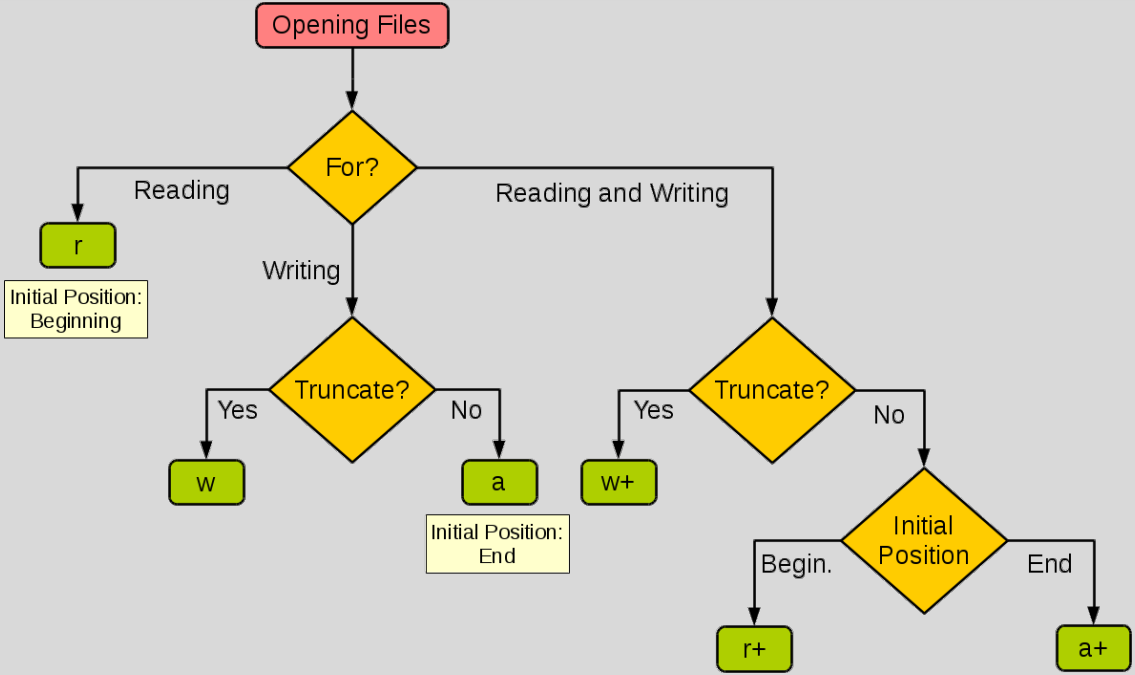
File IO functions in <stdio.h>

Function	Description
char *fgets(char *str, int n, FILE *fp)	Read up to (n - 1) characters from a line of a file then store them and the null character into the string str.
int fputs(const *str, FILE *fp)	Write the string str to a file. Return a non-negative integer on success or EOF on failure.
int fscanf(FILE *fp, const char *format, …)	Read formatted input from a file. Return the number of input items successfully matched and assigned, or zero in the event of an early matching failure.
int fprintf(FILE *fp, const char *format, …)	Write formatted output to a file. Return the total number of characters successfully written, or a negative number if failure.

Modes for fopen()

Mode	Description
r	Open a file for reading.
w	Open a file for writing.
a	Open a file for appending, i.e., writing at the end of the file. Create a new file if it doesn't exist.
r+	Open a file for reading and writing.
w+	Open a file for writing and reading. Overwrite the file if it exists.
a+	Opens a file for reading (from the beginning) and writing (at the end). Create a new file if it doesn't exist.

Modes for fopen()



Example: Write characters to file

```c
#include <stdio.h>

int main() {
    // Open file
    FILE *fp = fopen("test.txt", "w");
    if (fp == NULL) {
        printf("Cannot open file\n");
        return -1;
    }

    // Write some characters to file
    for (int ch = '0'; ch <= 'z'; ch++) {
        fputc(ch, fp);
    }

    // Close file
    fclose(fp);

    return 0;
}
```

Example: Read characters from file

```c
#include <stdio.h>

int main() {
    // Open file
    FILE *fp = fopen("test.txt", "r");
    if (fp == NULL) {
        printf("Cannot open file\n");
        return -1;
    }

    // Read each characters from file
    for (int ch = fgetc(fp); ch != EOF; ch = fgetc(fp)) {
        putchar(ch);
    }

    // Close file
    fclose(fp);

    return 0;
}
```

Example: Write and read characters from file

```c
#include <stdio.h>

int main() {
    // Open file
    FILE *fp = fopen("test.txt", "w+");
    if (fp == NULL) {
        printf("Cannot open file\n");
        return -1;
    }

    // Write some characters to file
    for (int ch = '0'; ch <= 'z'; ch++) {
        fputc(ch, fp);
    }

    // Read each character from file and print it
    rewind(fp);
    for (int ch = fgetc(fp); ch != EOF; ch = fgetc(fp)) {
        putchar(ch);
    }

    // Close file
    fclose(fp);
```

Example: Write strings to file

```c
#include <stdio.h>

int main() {
    // Open file
    FILE *fp = fopen("test.txt", "w");
    if (fp == NULL) {
        printf("Cannot open file\n");
        return -1;
    }

    // Write some strings to file
    fputs("Goodday everyone!\n", fp);
    fputs("It's week 11 now\n", fp);

    // Close file
    fclose(fp);

    return 0;
}
```

Example: Write and read strings from file

```c
#include <stdio.h>

int main() {
    // Open file
    FILE *fp = fopen("test.txt", "w+");
    if (fp == NULL) {
        printf("Cannot open file\n");
        return -1;
    }

    // Write some strings to file
    fputs("Goodday everyone!\n", fp);
    fputs("It's week 11 now\n", fp);

    // Read strings from file and print them
    rewind(fp);
    char s[20];
    fgets(s, 19, fp);
    printf("%s", s);
    fgets(s, 19, fp);
    printf("%s", s);

    // Close file
    fclose(fp);

    return 0;
}
```

Example: Write and read strings, ints, floats from file

```c
#include <stdio.h>

int main() {
    // Open file
    FILE *fp = fopen("test.txt", "w+");
    if (fp == NULL) {
        printf("Cannot open file\n");
        return -1;
    }

    // Write some strings, ints, floats to file
    fprintf(fp, "Goodday everyone!\n");
    int num1 = 5, num2 = 8;
    fprintf(fp, "%d + %d = %d\n", num1, num2, num1 + num2);
    float float1 = 5, float2 = 8;
    fprintf(fp, "%f * %f = %f\n", float1, float2, float1 * float2);

    // Read some strings, ints, floats from file
    char s[80];
    int n1, n2, n3;
    float f1, f2, f3;
    rewind(fp);
    fscanf(fp, "%[^\n]s", s);
    printf("%s\n", s);
    fscanf(fp, "%d + %d = %d", &n1, &n2, &n3);
    printf("%d %d %d\n", n1, n2, n3);
    fscanf(fp, "%f * %f = %f", &f1, &f2, &f3);
    printf("%f %f %f\n", f1, f2, f3);

    // Close file
    fclose(fp);

    return 0;
}
```

Command line arguments

- The full format of the main() function is
  - **int main(int argc, char const *argv[])**
  - **argc**: argument counter, i.e. number of arguments
  - **argv**: array of arguments
- We can pass the arguments when running a program such
  - **./program one two three**
- Then use the values of the arguments in the program

Example: Command line arguments

```c
#include <stdio.h>

int main(int argc, char const *argv[]) {
    printf("argc: %d\n", argc);
    for (int i = 0; i < argc; i++) {
        printf("argv[%d]: %s\n", i, argv[i]);
    }

    return 0;
}
```

Example: Command line arguments

```c
/*
  Compile:  gcc hello.c -o hello
  Run:      ./hello name1 name2 name3 ...
 */
#include <stdio.h>

int main(int argc, char const *argv[]) {
    for (int i = 1; i < argc; i++) {
        printf("Hello %s\n", argv[i]);
    }

    return 0;
}
```

Problem

Ask user for the name of a text file then print out its contents to the screen.

References

1. Fresh2Refresh, C Programming Tutorial, 2020.
2. StudyTonight, C Programming Language Tutorial, 2020.